

Pruning Basics: Removing Redundancy to Slim Down Neural Networks

A practical guide to making your models faster, smaller, and more efficient through strategic weight removal

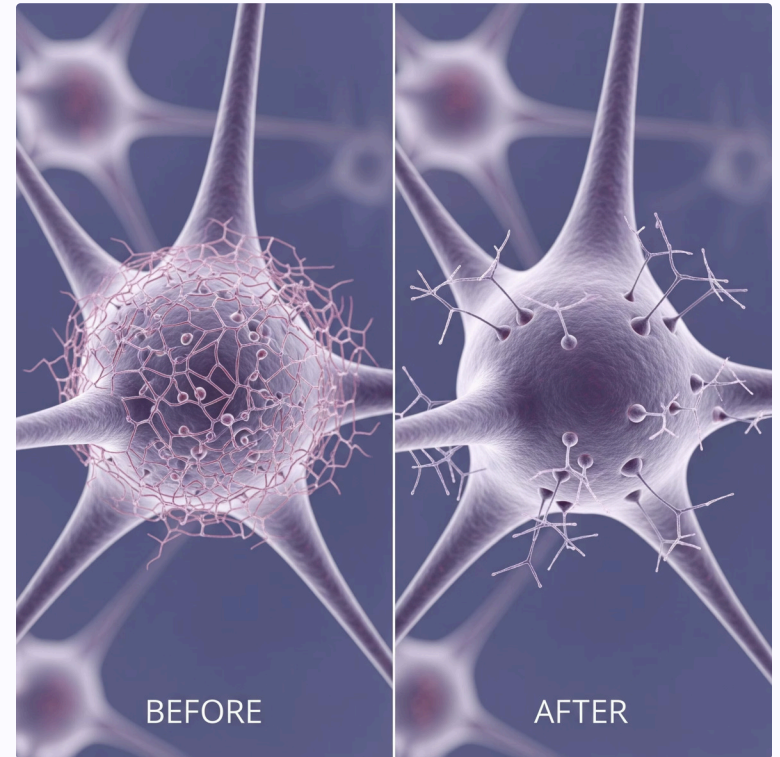


What Is Pruning?

Pruning is a model compression technique that **removes unnecessary weights or neurons** from a neural network, with the goal of reducing model size, memory footprint, and computational requirements.

Much like biological synapse pruning in the human brain, neural network pruning maintains critical pathways while eliminating redundant ones.

When done carefully, a pruned model maintains most of its original accuracy while becoming significantly more efficient.



Why Prune?



Eliminate Redundancy

Modern neural networks are heavily **over-parameterized**, containing many unnecessary connections that contribute little to performance



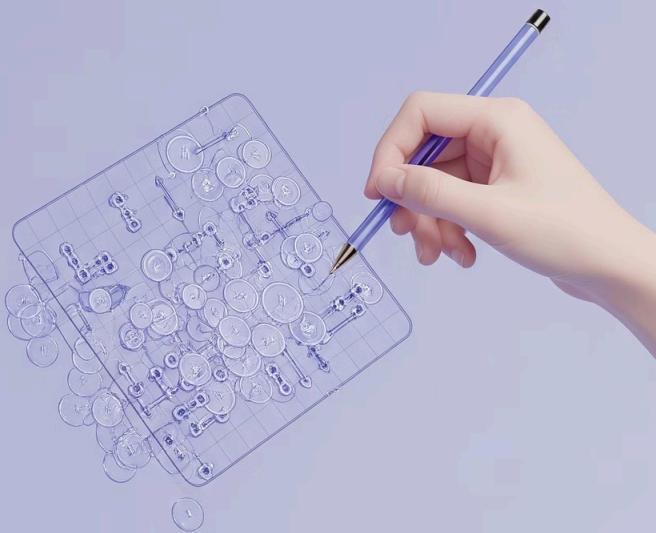
Faster Inference

Smaller models deliver **lower latency** and higher throughput, critical for real-time applications and production environments



Enable Edge Deployment

Reduced **storage & memory requirements** make it possible to deploy models on resource-constrained edge & mobile devices



Types of Pruning

Unstructured Pruning

Removes individual weights (sets them to 0) based on importance criteria

Structured Pruning

Removes entire structural components (filters, channels, layers)

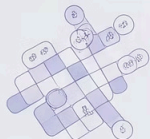
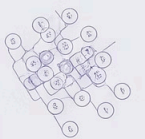
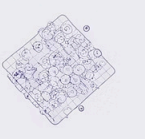
Global vs Local

Global pruning removes weights across the entire model; local pruning operates on specific layers

Dynamic Pruning

Adapts pruning patterns during inference based on input data characteristics

Pruning Approches Entire Filters Eliminated Pruning Approches

		VS		
Individual Weights being removed	Global vs Pruning		V = Local Pruning	Dynamic Pruning paseris on input

Unstructured Pruning

Unstructured pruning selectively zeroes out individual weights in the model, typically targeting those with the smallest magnitude as they contribute least to the output.

Characteristics:

- Can achieve very **high compression ratios** (>90% sparsity possible)
- Preserves model architecture
- Minimal accuracy impact at moderate sparsity levels
- Requires specialized sparse matrix operations for actual speedup



Without specialized sparse kernels, unstructured pruning may not deliver practical speedups despite size reduction



Structured Pruning



Remove Entire Filters

Eliminates complete filters, channels, or attention heads



Hardware Efficiency

Maintains dense computation patterns compatible with GPU/TPU acceleration



Deployment-Friendly

Easily integrates with production runtimes like ONNX and TensorRT

⚠ Caution: Structured pruning typically causes more aggressive accuracy drops than unstructured pruning at the same compression ratio. Balance is critical!

Example: PyTorch Unstructured Pruning

```
import torch
import torch.nn.utils.prune as prune

# Create a simple linear layer
model = torch.nn.Linear(128, 64)

# Prune 30% of weights with lowest L1 norm
prune.l1_unstructured(model, name="weight", amount=0.3)

# Inspect the pruning mask (1 = kept, 0 = pruned)
print(model.weight_mask) # binary mask of pruned weights

# Make pruning permanent (optional)
prune.remove(model, 'weight')
```

This example demonstrates how to apply unstructured pruning to a linear layer using PyTorch's built-in pruning utilities. The `weight_mask` shows which weights are retained (1) vs pruned (0).

Workflow for Pruning

Train Baseline Model

Start with a fully-trained, high-performing model as your baseline

Apply Pruning Strategy

Implement unstructured or structured pruning based on your deployment constraints

Fine-tune the Model

Retrain on training data to recover accuracy lost during pruning

Export for Deployment

Convert to inference-optimized format (ONNX, TensorRT, TFLite, etc.)

Benchmark Performance

Measure latency, throughput, memory usage and accuracy on target hardware

Benefits of Pruning

Dramatic Size Reduction

Achieve 10-90% reduction in model size depending on architecture and pruning approach

Faster Inference

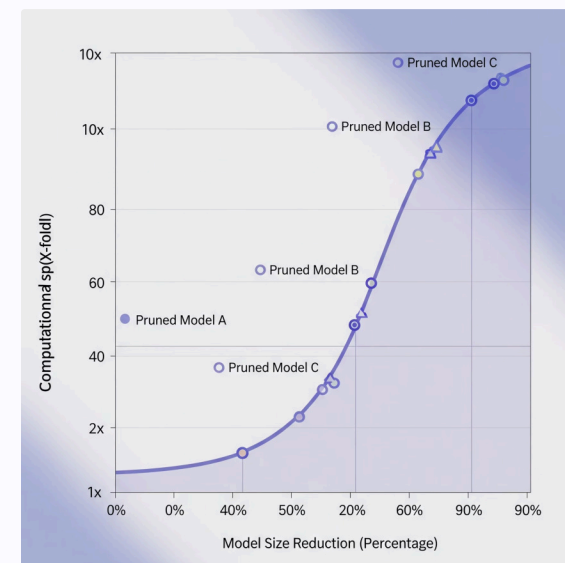
Structured pruning delivers real-world latency improvements for production workloads

Energy Efficiency

Lower computation requirements translate to energy savings in large-scale deployments

Complementary Technique

Combines effectively with **quantization** and knowledge distillation for compound efficiency gains



Pruning yields a favorable trade-off between model size, speed, and accuracy when applied correctly

Limitations and Challenges

Accuracy Degradation

Aggressive pruning can significantly reduce model performance, especially on out-of-distribution data

Hardware Compatibility

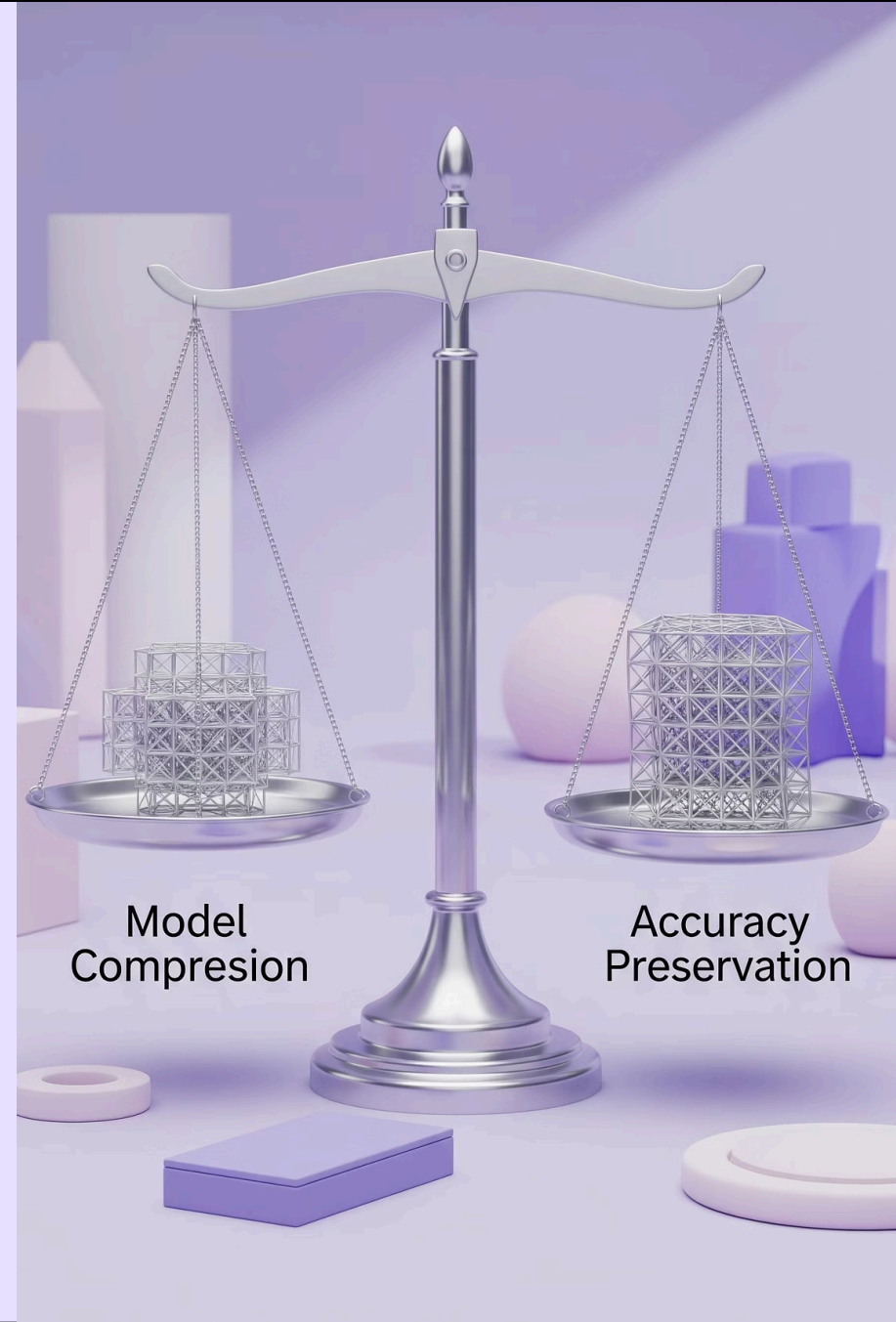
Unstructured pruning often yields little to no speedup without specialized sparse matrix libraries or hardware support

Increased Training Complexity

Post-pruning fine-tuning adds time and complexity to the training pipeline

Challenging Trade-offs

Balancing pruning percentage against accuracy requires careful experimentation specific to each model and task



Best Practices for Effective Pruning

Start Conservative

Begin with 10-20% pruning and gradually increase, monitoring accuracy impacts at each step

Prioritize Structure

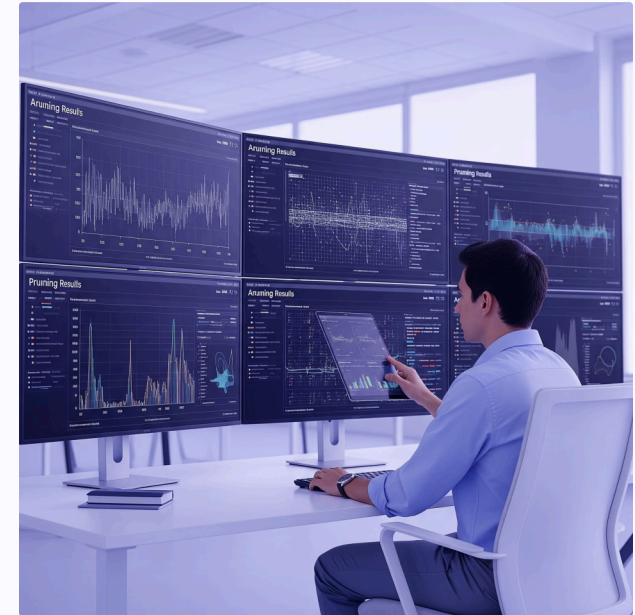
Prefer structured pruning for real-world deployment efficiency unless compression ratio is the only concern

Fine-tune Strategically

Always fine-tune after pruning, potentially with a lower learning rate and fewer epochs than initial training

Measure What Matters

Track end-to-end metrics like latency and throughput on target hardware, not just theoretical parameter counts



- ❏ Different layers have different sensitivity to pruning. Conv layers near the input typically need more parameters than those deeper in the network.

Key Takeaways

Pruning is essential for efficient deep learning



Pruning strategically removes redundant weights and connections to streamline neural networks



Structured pruning delivers the most practical hardware acceleration benefits despite being more challenging



Post-pruning fine-tuning is critical for maintaining accuracy and model stability



When combined with other compression techniques, pruning enables efficient deep learning at scale, from cloud to edge

